ECEN 248 - Lab Report

Post Lab #9

An Introduction to High-Speed Addition

Section: 524

Ramya Kannan

UIN: 635002011

11/20/2025

TA: Reed Clardy

Objective

The objective of this lab is to design and implement carry-lookahead addition circuits that compute arithmetic carries in parallel for faster operation than ripple-carry adders. We will create a 4-bit carry-lookahead adder and a 16-bit two-level carry-lookahead adder using dataflow and structural Verilog, simulate them in Xilinx Vivado with gate delays, and compare their propagation delays and gate counts against ripple-carry adders.

Design

We created three sub-modules using dataflow Verilog. The generate_propagate_unit computed G = X & Y and P = X ^ Y with 2ns delays. The carry_lookahead_unit computed all four carries in parallel using expressions from the lab manual with 4ns delays per carry equation. The summation_unit computed S = P ^ C with 2ns delay.

Then we used them in the carry_lookahead_4bit using structural Verilog, instantiating GPU, CLA, and SU modules. We created a C_sum vector by doing {C[3:1], Cin} to provide the proper carry inputs to the summation unit. It was tested with the cla_4bit_tb.v by adjusting test bench timing to measure actual propagation delay.

Then for part 2 of the experiment, we needed to implement the 16-bit two-level carry lookahead adder. First we created the block_carry_lookahead_unit, which outputs block-level Gstar and Pstar signals using expressions from the lab manual, along with internal carries C[3:1]. We then expanded the generate/propagate and summation units to 16—bit width.

The carry_lookahead_16bit module used structural Verilog to instantiate: one 16-bit generate_propagate_unit, four block_carry_lookahead_units (one per 4-bit block), one carry_lookahead_unit to generate block carries, and one 16-bit summation unit. We added 2ns delays to all submodules and calculated theoretical delay as 8 gate delays = 16ns. We tested this module with the cla-16bit_tb.v and measured the actual propagation delay.

Results

The 4-bit carry_lookahead adder passed all tests in the behavioral simulation. After adding 2ns gate delays, we measured the propagation delay as 8ns by incrementing test bench delays until all tests passed. The delay was found to be 8ns total. Gate Count (from the prelab) : **32 gates** (GPU (8) + CLAU (20) + SU (4) = 8).

The 16-bit two-level carry-lookahead adder passed all tests. With gate delays, we measured the propagation delay as 16ns. THe theoretical calculation from the prelab was 8 gate delays: GPU(1) + BCLAU Gstar/Pstar(2) + CLAU block carries(2) + BCLAU remaining carries(2) + SU(1) = 8 gate delays, which equals 2ns + 4ns + 4ns + 4ns + 2ns = 16ns. The measured delay matched the calculated delay. Gate count from prelab: **148 gates** (GPU: 32 gates, 4 BCLAUs: 80 gates, CLAU: 20 gates, SU: 16 gates).

Conclusion

This lab successfully designed a carry-lookahead adder demonstrating significant speed improvements over ripple-carry adders. The 4-bit CLA achieved 8ns delay compared to 16ns for

a 4-bit ripple-carry adder, making it twice as fast. The 16-bit CLA achieved 16ns delay compared to 64ns for a 16-bit ripple-carry adder, making it four times faster. This performance came at the cost of increased gate count. The 4-bit CLA required 32 gates versus 20 gates for ripple-carry, while the 16-bit CLA required 148 gates versus 80 gates for ripple-carry. The results demonstrate the performance advantage of parallel carry computation for wider operands.

Post-Lab Deliverables

1. Source Code

Figure 1. Carry Lookahead 4bit

```
`timescale 1ns / 1ps
`default_nettype none

module carry_lookahead_4bit(Cout, S, X, Y, Cin);

//ports are wires as we will use structural
    output wire Cout; //C4 for a 4-bit adder
    output wire [3:0] S; //final 4-bit sum vector
    input wire [3:0] X, Y; //the 4-bit addends
    input wire Cin; //input carrry

    wire [3:0] G, P; //generate and propagate vectors
    wire [4:1] C;
    assign Cout = C[4];
    wire [3:0] C_sum; //carry vector for sum
    //now create a carry vector for summation unit
    assign C_sum = {C[3:1], Cin};

    generate_propagate_unit GPU(G, P, X, Y);

    //instantiate the CLA unit
    carry_lookahead_unit CLA(C, G, P, Cin);

    //instantiate the summation unit
    summation_unit SU(S, P, C_sum);
endmodule
```

Figure 2. Generate Propagate Units

```
`timescale 1ns / 1ps
`default_nettype none

module generate_propagate_unit(G, P, X, Y);
    //ports are wires, dataflow
    output wire [15:0] G, P;
    input wire [15:0] X, Y;
    //generate function: g_i = x_i * y_i
    assign #2 G = X & Y;
    //propagate function = p_i = x_i XOR y_i
    assign #2 P = X ^ Y;
endmodule
```

Figure 3. Summation Unit

```
`timescale 1ns / 1ps
`default_nettype none

module summation_unit(S, P, C);
    //ports are wires because we willuse dataflow
    output wire[15:0] S; //sum vector
    input wire [15:0] P, C; //propagate and carry vectors

    //sum bits = s_i = p_i XOR c_i
    assign #2 S = P ^ C;
endmodule
```

Figure 4. Block Carry Lookahead Unit

```
`timescale 1ns / 1ps
`default_nettype none

module block_carry_lookahead_unit(Gstar, Pstar, C, G, P, C0);

    /*
     * ports are wires because we will use dataflow
     */
    output wire Gstar, Pstar;   // block generate and propagate
    output wire [3:1] C;        // C3, C2, C1
    input wire [3:0] G, P;      // generates and propagates
    input wire C0;              // input carry

    // Internal carries (same as regular CLA unit)
    // c_1 = g_0 + p_0*c_0
    assign #4 C[1] = G[0] | (P[0] & C0);

    // c_2 = g_1 + p_1*g_0 + p_1*p_0*c_0
    assign #4 C[2] = G[1] | (P[1] & G[0]) | (P[1] & P[0] & C0);

    // c_3 = g_2 + p_2*g_1 + p_2*p_1*g_0 + p_2*p_1*p_0*c_0
    assign #4 C[3] = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0]) | (P[2] & P[1] & P[0] & C0);

    // Block generate: G* = g_3 + p_3*g_2 + p_3*p_2*g_1 + p_3*p_2*p_1*g_0
    // This is equation (12) from the lab manual
    assign #4 Gstar = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1]) | (P[3] & P[2] & P[1] & G[0]);

    // Block propagate: P* = p_3*p_2*p_1*p_0
    // This is equation (13) from the lab manual
    assign #4 Pstar = P[3] & P[2] & P[1] & P[0];

endmodule
```

Figure 5. 16-bit Carry Lookahead

```
`timescale 1ns / 1ps
`default_nettype none

/*
 * This is the top-level module for a 16-bit
 * 2-level carry-lookahead adder
 */
module carry_lookahead_16bit(Cout, S, X, Y, Cin);

    /*
     * ports are wires as we will use structural
     */
    output wire Cout;           // C16 for a 16-bit adder
    output wire [15:0] S;       // final 16-bit sum vector
    input wire [15:0] X, Y;     // the 16-bit addends
    input wire Cin;             // input carry!

    /*
     * intermediate nets
     */
    wire [16:0] C;              // 17-bit carry vector
    wire [15:0] P, G;           // generate and propagate vectors
    wire [3:0] Pstar, Gstar;    // block gens and props

    /*
     * hook up input and output carry
     */
    assign C[0] = Cin;          // Connect input carry to C[0]
    assign Cout = C[16];        // Connect C[16] to output carry

    /*
     * sub-modules
     */
    // Instantiate the 16-bit generate/propagate unit
    generate_propagate_unit GPU(
        .G(G),
        .P(P),
        .X(X),
        .Y(Y)
    );
```

```
    block_carry_lookahead_unit BCLAU0(
        .Gstar(Gstar[0]),
        .Pstar(Pstar[0]),
        .C(C[3:1]),
        .G(G[3:0]),
        .P(P[3:0]),
        .C0(C[0])
    ); // a little more verbose, but much easier to follow!

    // Instantiate BCLAU 1
    block_carry_lookahead_unit BCLAU1(
        .Gstar(Gstar[1]),
        .Pstar(Pstar[1]),
        .C(C[7:5]),
        .G(G[7:4]),
        .P(P[7:4]),
        .C0(C[4])
    );

    // Instantiate BCLAU 2
    block_carry_lookahead_unit BCLAU2(
        .Gstar(Gstar[2]),
        .Pstar(Pstar[2]),
        .C(C[11:9]),
        .G(G[11:8]),
        .P(P[11:8]),
        .C0(C[8])
    );

    // Instantiate BCLAU 3
    block_carry_lookahead_unit BCLAU3(
        .Gstar(Gstar[3]),
        .Pstar(Pstar[3]),
        .C(C[15:13]),
        .G(G[15:12]),
        .P(P[15:12]),
        .C0(C[12])
    );
```

```
    carry_lookahead_unit CLAU(
        .C({C[16], C[12], C[8], C[4]}),
        .G(Gstar),
        .P(Pstar),
        .C0(C[0])
    );

    // Instantiate the 16-bit summation unit
    summation_unit SU(
        .S(S),
        .P(P),
        .C(C[15:0])
    );

endmodule
```

2. Waveform

Figure 6. Waveform 4-bit

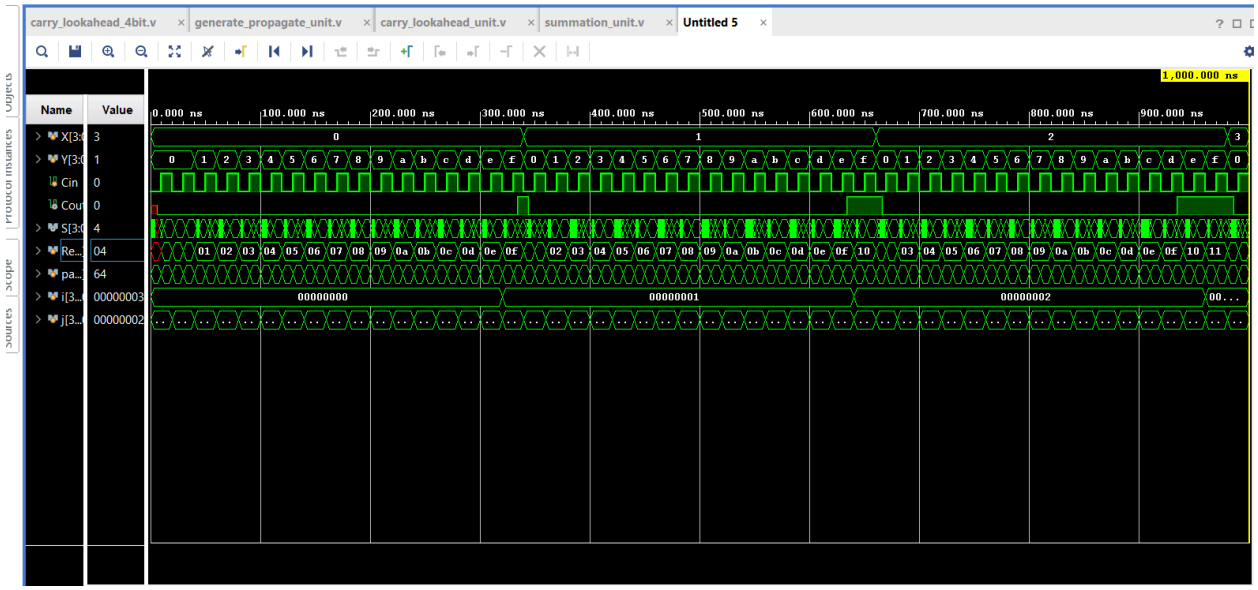
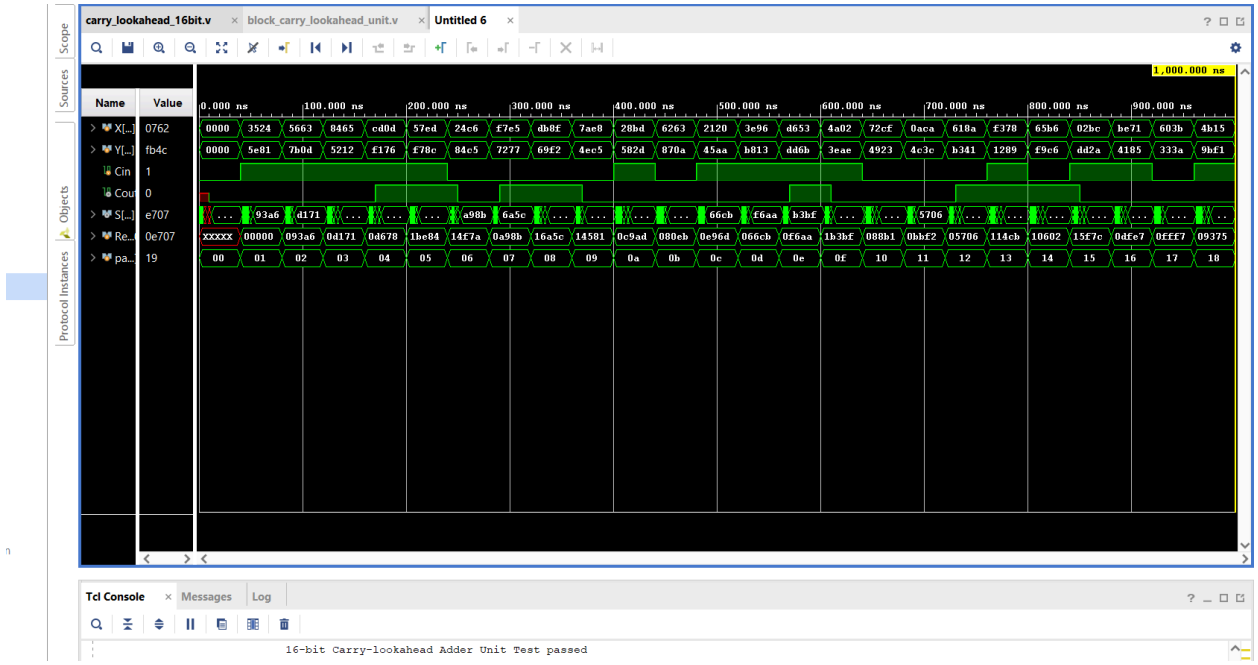


Figure 7. Waveform 16-bit



3.

a. The change in gate delay to 2ns: With 2ns per gate and 8 gates in the critical path, the worse-case delay is 16ns.

b. The propagation delay for the 16-bit CLA: The measured propagation delay is 16ns

c. Correlation: Yes, they correlate exactly. The measured delay of 16ns matches the prelab calculation of 8 gates * 2ns = 16ns.

4.

The 16-bit CLA uses 148 gates, the 16-bit ripple-carry uses 80 gates. Even though the CLA has almost double the gate count, it is significantly faster because it computes carries in parallel rather than sequentially through a carry chain!

5.

The 4-bit CLA has 8ns delay while the 4-bit ripple-carry has 16ns delay. The CLA completes addition in half the time due to parallel carry generation

6.

The 16-bit CLA has 16ns delay while the 16-bit ripple-carry has 64ns delay. The CLA is 4 times faster, demonstrating the performance advantage of parallel carry computation for wider operands

7.

For ripple-carry adders, delay grows linearly with bit width: for the 4-bit = 16ns, 16-bit = 64ns.. For carry-lookahead adders, delay grows logarithmically: 4-bit = 8ns, 16-bit = 16ns. The CLA's hierarchical structures enable this logarithmic growth pattern, making it increasingly advantageous as bit width increases.

Student Feedback

1. What did you like most about the lab assignment and why? What did you like least about it and why?

It was quick and I liked how the sub modules were reused for the 16-bit.

2. Were there any sections of the lab manual that were unclear? If so, what was unclear? Do you have any suggestions for improving the clarity?

No, the lab manual was clear.

3. What suggestions do you have to improve the overall lab assignment?

No, I have no suggestions for this assignment.